

Presentation du Dossier Projet

Titre Professionnel

Administrateur Systemes DevOps

Bachelor Administrateur Système DevOps · Thomas Guillian

Sommaire du Projet

Ce dossier couvre l'ensemble du parcours de déploiement et d'automatisation de l'application **Pasta**, de l'infrastructure locale jusqu'à la mise en production dans le cloud. Chaque section correspond à une compétence clé du titre professionnel d'administrateur systèmes DevOps.

01

Présentation du projet et contexte

L'application Pasta, son origine, ses objectifs pédagogiques et l'environnement de travail exploré.

ċċ

Automatiser le déploiement d'une infrastructure

Hyper-V, Terraform, GitLab CI/CD, Railway, Oracle Cloud et Google Cloud Platform....

03

Gérer les containers

Docker, Environnements de test, Volumes, Ansible.

ċç

Exploiter une solution de supervision

Uptime Kuma, Métriques GCP, Alertes mail, Administration mobile et statistiques de service.

05

Échanges professionnels en anglais

Expériences professionnels, Travail interallié multilingue, Recherche d'emploi.

Présentation du Projet : Pasta / Meal Planner 🍝

***Pasta** est une application web conçue pour évoluer par incrémentation : chaque nouvelle fonctionnalité importante donne lieu à la création d'une nouvelle branche Git. L'outil permet à l'utilisateur de sélectionner une recette de pâtes, de définir un nombre de convives, et reçoit en retour la liste minimale des ingrédients nécessaires à la recette.*

Pourquoi cette application ?

*L'application est volontairement simple : pas de base de données complexe, pas d'authentification lourde. Cette simplicité est un **atout pédagogique** — elle permet de concentrer l'effort sur les outils DevOps eux-mêmes plutôt que sur du développement applicatif.*

L'idée est née d'une observation du quotidien : mon entourage ne sait généralement pas quoi manger, et encore moins quoi acheter au supermarché. Un problème simple, une solution simple, un terrain d'expérimentation idéal.

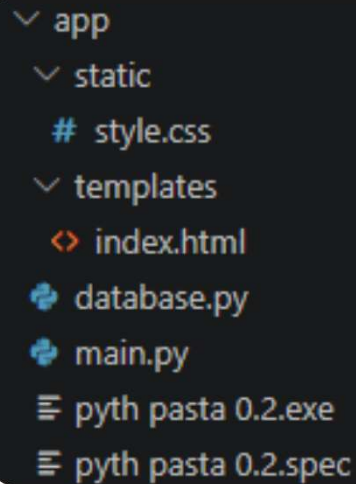
Contexte & Titre Professionnel

*Ce projet s'inscrit dans le cadre du titre professionnel d'**Administrateur Systèmes DevOps**. L'application a volontairement été choisie comme support évolutif afin de démontrer les compétences acquises à chaque étape, sans que la complexité applicative ne devienne un frein.*

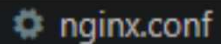
Pour moi, issu des forces armées où le code se limitait essentiellement à des scripts PowerShell et Bash, ce projet représente aussi un retour progressif et personnel dans le monde du développement logiciel — avec Python comme terrain d'exploration.

- Application Flask exposée via Nginx
- Conteneurisée avec Docker
- Hébergée sur GCP via Terraform + Ansible

Application Flask .py exposé avec nginx



```
▼ app
  ▼ static
    # style.css
  ▼ templates
    <> index.html
  database.py
  main.py
  pyth pasta 0.2.exe
  pyth pasta 0.2.spec
```



```
⚙️ nginx.conf
```

Nginx agit comme un reverse proxy qui écoute le port 80 et 443 pour renvoyer sur l'application qui expose le port 8000.

Le fichier Nginx.conf est alimenté par des certificats ssl letsencrypt

```
[Client] → :80/:443 → [Nginx] → :8000 → [Flask]
```

GET affiche les plats depuis database.py via SQLite

POST recoit les plats effectue les calculs et retourne un resultat

(en fonction de la branche)

- Docker build -t pasta-web:local

- Docker run -rm -p 8000:8000 pasta-web:local

- http://127.0.0.1:8000 / http://localhost

Conteneurisation avec docker

Copie le contenu de requirements.txt afin de mettre à jour les dépendances

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

COPY ./app /app

ENV PORT 8000
EXPOSE $PORT

CMD ["python", "main.py"]
```

```
Flask==2.3.3
requests==2.31.0
```

- Docker build -t pasta-web:local
- Docker run -rm -p 8000:8000 pasta-web:local
- Docker compose up --build
- Docker compose down

Automatiser la Création de Serveurs

Plusieurs approches ont été explorées pour automatiser la création d'infrastructure, allant de la virtualisation locale sous Hyper-V jusqu'au provisionnement cloud via Terraform.

Hyper-V & Scripts PowerShell

La première approche consiste à utiliser des scripts **.ps1** envoyant à Hyper-V une configuration permettant la création d'une VM (une lubuntu légère), sans installation ni configuration. C'est une méthode simple mais puissante : une bonne suite de fichiers **.ps1**, **.bat** ou **.sh** peut accomplir de grandes choses — comme en témoigne leur usage dans l'industrie de la défense depuis 30 ans.

Les VMs locales présentent un avantage réseau important : elles peuvent rester sur un réseau interne fermé (pas de sortie Internet), favorisant le travail collaboratif sécurisé. Une VM dédiée peut jouer le rôle de proxy pour les autres, masquant leurs IP internes.

Terraform sur Oracle Cloud (Free Tier)

Malgré de nombreuses tentatives, le free tier Oracle ne disposait pas de capacités suffisantes pour héberger l'application. Le travail fourni est documenté, mais **l'hébergeur final a été migré vers Google Cloud Platform**, une décision rapide qui illustre la portabilité des configurations Terraform.

Terraform sur Google Cloud Platform

GCP s'est révélé nettement plus accessible qu'Oracle — « environ 2 jours et 1 minute 50 secondes plus rapide pour un même cerveau ». Les fichiers **.tf** permettent de :

- Créer la VM et la rendre accessible depuis la console GCP
- Injecter les clefs SSH directement via la configuration (prérequis sécurité)
- Paramétrer le réseau, les règles de pare-feu et les métadonnées

Une fois la VM provisionnée par Terraform, **Ansible prend le relais** pour la configuration applicative. Ce duo est l'épine dorsale du déploiement automatisé du projet.

📄 La structure des fichiers **.tf** a évolué au fil du projet — preuve que l'Infrastructure as Code s'affine avec l'expérience.

Deploiement Terraform

```
resource "google_compute_firewall" "allow_http_ssh" {  
  name     = "allow-http-ssh"  
  network  = "default"  
  allow {  
    protocol = "tcp"  
    ports    = ["22", "80", "443", "3001"]  
  }  
  source_ranges = ["0.0.0.0/0"]  
}
```

```
provider "google" {  
  project = "project-6b19dd51-06fd-4946-8a6"  
  region  = "us-central1"  
  zone    = "us-central1-a"  
}
```

```
resource "google_compute_instance" "pasta-vm" {  
  name         = "pasta-vm"  
  machine_type = "e2-micro"  
  zone        = "us-central1-a"  
  
  boot_disk {  
    initialize_params {  
      image = "ubuntu-os-cloud/ubuntu-2204-lts"  
      size  = 30  
    }  
  }  
}
```

Google reserve une IP fixe dans la region demandée.

Les ports ouverts ;

22 SSH Ansible

80 HTTP Nginx

443 Nginx et Let's Encrypt

3001 UpTime Kuma

Machine e2-micro (disponible free tier)

Une VM Ubuntu avec 30 go de stockage

CI/CD : GitLab, Runner & Railway

L'automatisation du déploiement s'articule autour de deux axes complémentaires :

le pipeline GitLab CI pour les tests et la validation du code, et Railway pour le déploiement cloud simplifié.

GitLab CI — Pipeline de tests

Chaque `git push` déclenche l'exécution de stages définis dans le fichier `.gitlab-ci.yml`. Si un test échoue, le pipeline s'arrête — évitant ainsi de passer d'un service fonctionnel à une interruption de production. L'idée explorée : monter un conteneur en live à l'issue des tests pour valider les modifications en temps réel.

Railway — Déploiement PaaS

À partir d'un `Dockerfile` et d'une branche GitHub, Railway met en production l'application tout en effectuant des tests préliminaires. Il offre un FQDN propre, des métriques serveur détaillées, l'historique des commits pour le rollback, et une interface **très admin-friendly**. Idéal pour les petits besoins avec un coût maîtrisé.

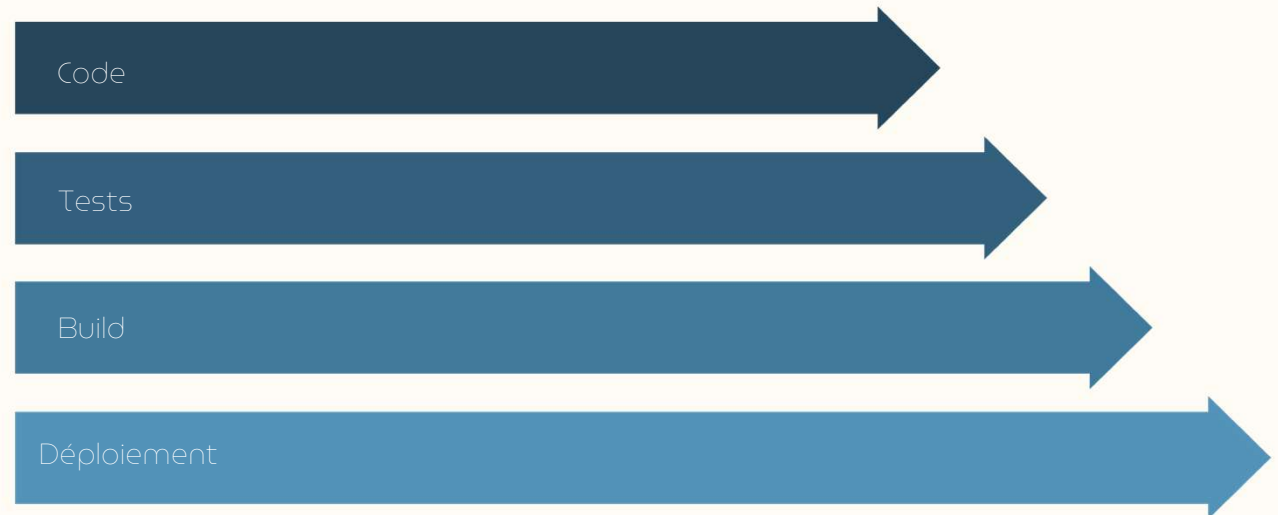
CI/CD : QitLab, Runner & Railway

Le Runner & Docker-in-Docker

*Certains stages nécessitent un Runner auto-hébergé. La complexité surgit avec le service **Docker-in-Docker** : le Runner exécute les tests dans un conteneur qui se monte, effectue les tests, puis se démonte. Monter un conteneur dans un conteneur — une véritable « inception ». La solution de déploiement automatique en production via cette méthode n'a pas abouti, mais l'expérience acquise est précieuse.*

Terraform + Ansible — Le duo production

*La combinaison finale retenue : **Terraform crée l'infrastructure, Ansible la configure**. Avec une poignée de commandes (`git push`, `gcloud auth`, `terraform apply`, `ansible-playbook...`), une infrastructure complète se déploie plus rapidement qu'à la main — et de manière reproductible.*



Ce pipeline illustre la philosophie DevOps : chaque étape est automatisée, tracée et reproductible. Un fichier `gitlab-ci.yml` bien configuré est le chef d'orchestre qui coordonne l'ensemble du processus de livraison.

Sécuriser l'Infrastructure

Même pour une application de petite taille, la sécurité est une préoccupation constante. La simplicité de l'application (pas de base de données, pas d'authentification complexe) réduit la surface d'attaque — mais ne dispense pas d'une bonne hygiène d'administration.

HTTPS via nip.io

*Ajout d'un certificat SSL via **nip.io** permettant une connexion sécurisée en HTTPS, sans avoir à monter une PKI interne avec un serveur de certificats racine et subalterne — une approche plus ergonomique et efficace pour un contexte de démonstration.*

Clefs SSH & Accès restreint

*L'authentification par clef SSH est un prérequis injecté directement via Terraform. Les données et le travail ne sont accessibles et modifiables que par les personnes autorisées — principe de **moindre privilège** appliqué à la gestion des accès.*

Règles Cloud Provider

Utilisation des configurations de règles d'administration fournies par l'hébergeur cloud (GCP) : pare-feu, règles d'entrée/sortie, restriction des ports non exposés. Une organisation très restreinte au niveau des différents services limite l'exposition en cas de compromission.

Vecteurs d'attaque identifiés

Les risques résiduels identifiés : vulnérabilité potentielle du Nginx (version non à jour), et bruteforce SSH (long et fastidieux). L'absence de base de données élimine les risques d'injection SQL ou d'exfiltration de données sensibles.

Gestion des Containers & Stockage

La conteneurisation avec Docker est au cœur de la portabilité du projet. Elle permet d'isoler l'environnement applicatif, d'embarquer les dépendances, et de garantir un comportement identique en local, en test et en production.

Docker & Isolation de l'environnement

*Le conteneur joue le rôle de serveur hébergeant le code et ses dépendances. Le repository se suffit à lui-même : il contient le Dockerfile, le code applicatif et la configuration. Cette autonomie est **idéale pour le cloud** — un simple docker build suivi d'un docker run suffit à démarrer le service.*

Volumes Persistants

*La création de volumes Docker garantit la **persistance des données** malgré les arrêts, redémarrages ou mises à jour du conteneur ou de la VM.*

Environnements de test avec Railway

*Railway permet de créer plusieurs environnements distincts, chacun lié à une branche Git spécifique. Chaque environnement est un doublon isolé permettant de tester les modifications sans impacter la production. Cette approche reproduit le modèle **dev / staging / production** classique.*

Git & GitLFS pour la gestion des fichiers

*GitHub sert de référentiel principal et permet, à l'instar d'un snapshot ou d'une sauvegarde, de revenir à n'importe quel état antérieur du projet. Sauf pour les fichiers volumineux (comme les fichiers générés par terraform apply), **Git Large File Storage (GitLFS)** crée un pointeur distant évitant les limitations de taille (100 MB par fichier). Les fichiers inutiles au versionnement sont exclus via **.gitignore**.*

- ❑ *L'incrémentation est organisée en ordre numérique — les décimales représentent des variantes mineures comme la portabilité des clefs SSH ou l'ajout de fonctionnalités secondaires.*

Supervision avec Uptime Kuma & GCP Metrics

Exploiter une solution de supervision est une compétence essentielle pour garantir la pérennité des services. Deux niveaux de supervision complémentaires ont été mis en place : applicatif avec Uptime Kuma, et infrastructure avec les agents GCP.

Uptime Kuma — Supervision applicative

Uptime Kuma est un applicatif de supervision open-source auto-hébergeable qui centralise la surveillance de plusieurs services. Il offre :

- **Sondes** : vérification de la disponibilité des endpoints (HTTP, TCP, ping...)
- **Temps de réponse** en temps réel du serveur
- **Alertes** configurables avec seuils personnalisés
- **Notifications** (email, Telegram, Slack...) pour l'astreinte
- **Sauvegarde** de la configuration et collecte de statistiques en temps réel
- **Pages de statut** publiques pour communiquer les incidents

Métriques GCP — Supervision infrastructure

*Via un **agent installé sur la VM hébergée par GCP**, des métriques hardware sont collectées : CPU, mémoire, disque, réseau. Ces métriques complètent Uptime Kuma qui surveille la couche applicative — les deux niveaux sont **indispensables et complémentaires**.*

Administration mobile & Astreinte

*Une fois les alertes liées à une adresse mail de supervision avec les notifications activées, l'astreinte se fait dans les meilleures conditions. Il est également possible d'**administrer en SSH depuis un smartphone**, permettant de relancer les services à tout moment. Cette disponibilité permanente améliore la qualité de service globale.*

Railway intègre aussi ses propres métriques et l'historique des commits pour un rollback rapide en cas d'incident.



Alertes & Notifications

Configuration d'alertes sur seuils avec notifications email et mobile pour une réactivité optimale en cas d'incident.



Statistiques de service

Collecte continue des métriques applicatives (uptime, latence) et infrastructure (CPU, RAM, réseau) pour le suivi de la qualité de service.



Administration distante

SSH depuis smartphone, tableau de bord Railway et console GCP permettent une supervision et une intervention à tout moment, depuis n'importe où.

Pré-Requis a la supervision

Pour GCP:

Cela nécessite d'ajouter un agent dans le VM, suite à cela le monitoring Propose énormément de choses très pertinentes.

Pour RailWay:

Tout se situe à même le projet, est déjà pré-configuré un dashboard.

Pour UpTime Kuma :

*Il faut le renseigner dans le docker-compose.yml et dans le main.tf
Il propose une interface et un service très ergonomique et très qualitatif.*

▷ Run Service

uptime-kuma:

image: louislam/uptime-kuma:1

container_name: uptime-kuma

ports:

- "3001:3001"

volumes:

- uptime-kuma-data:/app/data

restart: unless-stopped

```
guillian_mt4@pasta-vm:~$ curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh  
sudo bash add-google-cloud-ops-agent-repo.sh --also-install
```

Échanges Professionnels en Anglais

Au-delà de la technique pure, ce dossier aborde également la dimension professionnelle internationale, nourrie par une expérience militaire significative au sein de missions OTAN.

Common Tenacity — Administrateur système interallié

*Durant cette mission OTAN, j'ai exercé les fonctions d'**administrateur système (SICF)** au profit de l'interallié. L'équipe technique était composée de 3 personnes : un administrateur système, un administrateur réseau, avec en support un groupe helpdesk d'une vingtaine de techniciens allemands. La langue de travail et de coordination était exclusivement **l'anglais technique** — Des comptes rendus d'incidents aux procédures d'escalade.*

Cold Response & Brilliant Jump — Norvège

Deux exercices supplémentaires en Norvège ont consolidé cette pratique de l'anglais professionnel dans un contexte hautement technique. Toutes les interactions — briefings, documentation, support technique — se déroulaient en anglais. La proximité des compétences entre techniciens de différentes nationalités créait un terrain d'échange naturellement professionnel et enrichissant.

*Ces expériences démontrent une capacité établie à **communiquer efficacement en anglais sur des réseaux professionnels**, dans des environnements contraignants et à fort enjeu opérationnel.*

❏ *Ces missions illustrent une dimension souvent sous-estimée du profil DevOps : la capacité à travailler en équipe multiculturelle, à documenter et à communiquer dans un anglais technique précis — compétence de plus en plus requise dans les environnements cloud et open-source.*

Conclusion & Bilan du Projet

Un projet complet, exigeant, et profondément formateur. L'application Pasta — aussi simple soit-elle — a servi de terrain d'expérimentation pour l'ensemble de la chaîne DevOps, de la ligne de code au déploiement cloud supervisé.

Infrastructure as Code

Terraform (GCP) provisionne l'infrastructure. Ansible configure la VM. Le tout reproductible en quelques commandes.

Conteneurisation

Docker isole et rend portable l'application. Volumes persistants, DockerFile et DockerCompose constituent une base solide.

CI/CD

GitLab CI avec Runner auto-hébergé. Railway pour le déploiement PaaS simplifié. Pipeline de validation avant mise en production.

Supervision

Uptime Kuma pour l'applicatif, agents GCP pour l'infrastructure. Des Alertes mail, Une administration mobile, Des métriques en temps réel.

Ce que le projet restera

*L'application sera hébergée sur **GCP jusqu'au 4 juillet 2026**, et sur Railway jusqu'en fin de semaine. Le repository GitHub restera public. Un second repository sera créé pour partager le travail comme exemple de référence pour d'autres étudiants.*

L'incrémentation par branches reflète une progression méthodique : chaque numéro de branche correspond à une étape, les décimales aux variantes mineures.

Ce que le projet apporte

*Au-delà des compétences techniques, ce projet a cultivé une **approche d'apprentissage par l'expérimentation** : tester, échouer, comprendre, corriger. Oracle Cloud n'a pas fonctionné ? Migration vers GCP. Le Docker-in-Docker n'a pas abouti ? La leçon reste gravée.*

Une infrastructure hébergée dans le cloud, montée par Terraform, configurée par Ansible, supervisée par Uptime Kuma — et déployable en quelques commandes. C'est précisément ce qu'on attend d'un administrateur systèmes DevOps.